ESCUELA COLOMBIANA DE INGENIERÍA

DESARROLLO ORIENTADO POR OBJETOS

Excepciones

2026-2 — Laboratorio 4/6

Responsables: José Luis García, Kevin David Quezada

PARTE I. Practicando con excepciones

A MDD y BDD con neoFlix

1. En su directorio descarguen los archivos contenidos en neoFlix.zip, revisen el contenido y estudien el diseño estructural de la aplicación (únicamente la zona en azul).
   R/=
   Son 6 clases, de las cuales se ven las relaciones entre ellas, los atributos y metodos de cada una, herencias
2. Expliquen por qué el proyecto no compila. Realicen las adiciones necesarias para lograrlo.
   R/=
   porque las excepciones no estan contruidas del todo, tienen solo un esqueleto

3. Dado el diseño y las pruebas documenten (si es necesario) y codifiquen el método raiting().
   R/=
   Se documentó con Javadoc y se codificó el método rating() de Series: retorna el promedio (división entera) de las valoraciones de sus episodios. Es fail-fast: la serie no sustituye ni ignora nada; la primera valoración que no se pueda calcular se propaga tal cual (VALUE_UNKNOWN si el episodio no tiene votos, DATA_ERROR si sus datos violan el invariante). Si la serie no tiene episodios, lanza CONTENT_EMPTY. Quedó verificado con las 4 pruebas dadas de SeriesTest (promedio 8 con las valoraciones 8, 8, 8, 9, 9, y las tres excepciones).

4. Dada la documentación y el diseño, codifiquen y prueben el método raiting(default).
   R/=

Se documentó, codificó y probó rating(int default_): promedio de las valoraciones de los episodios, sustituyendo cada valoración desconocida por default_ e ignorando (sin contarlas) las que tienen error en los datos. Si ningún episodio aporta valoración —serie vacía o todos con error— se lanza VALUE_UNKNOWN: sin ninguna referencia, la serie no puede 'inventar' un valor. Quedó probado con las pruebas propias de las sobrecargas (S-01…S-14).

5. Documenten, diseñen, codifiquen y prueben el método raiting(previous).
   R/=

   Se documentó, codificó y probó rating(boolean previous): sustituye cada valoración desconocida por el promedio (entero) de las valoraciones conocidas —las de los episodios anteriores si previous es true (promedio progresivo: cada episodio solo puede mirar hacia atrás), o las de toda la serie si es false (pre-escaneo)—. Serie vacía → CONTENT_EMPTY; sin valoraciones de referencia → VALUE_UNKNOWN; un error en los datos → DATA_ERROR (se propaga, no se sustituye). Probado con las pruebas S-01…S-14.

6. Propongan el cálculo, documenten, diseñen, codifiquen y prueben el método popularity(). (Definan reglas que obliguen a utilizar excepciones en sus cuatro momentos: propagar, lanzar y atender)
   R/=

   La popularidad de una serie es el promedio (división entera) de las popularidades conocidas de sus episodios, donde la de cada episodio es (terminados*100)/intentos. Un episodio sin intentos no es 'impopular': es 'no medido' y se ignora; si ninguno es medible, VALUE_UNKNOWN; serie vacía → CONTENT_EMPTY; y violar el invariante (terminados > intentos) → DATA_ERROR. Las reglas obligan a usar la excepción en sus cuatro momentos: (1) DECLARAR — throws NeoFlixException en las firmas de popularity(); (2) LANZAR — Audiovisual.popularity() la lanza al detectar la violación; (3) PROPAGAR — Series.popularity() agrega las de sus episodios y deja subir el error sin decidir nada; (4) ATENDER — data()/indicators() la capturan para degradar con elegancia ('*** Indicadores incompletos') sin romper el listado. Probado con S-15…S-21 de SeriesTest y E-01…E-10 de EpisodeTest.

PARTE II. Desarrollando neoFlix

A Revisando línea base

1. En su directorio descarguen los archivos contenidos en neoFlixMore.zip, revisen el contenido.
   a. ¿Cuántos archivos se tienen?

   R/=

   Tres archivos: Log, NeoFlix, NeoFlixGUI

   b. ¿Cómo están organizados?

   R/=

   A nivel de directorio se encuentran en \neoFlixMore\

   En detalle: los tres archivos vienen 'planos', sueltos en una sola carpeta de descarga (sin separarlos por paquete), aunque cada uno declara en su primera línea el paquete al que pertenece: Log y NeoFlix declaran package domain; y NeoFlixGUI declara package presentation.

   c. ¿Cómo deberían estar organizados?

   R/=

   En un paquete dentro de neoFlix

   En detalle: cada archivo dentro de la carpeta del paquete que declara (domain/ y presentation/) dentro del proyecto neoFlix; así lo organizan las herramientas (BlueJ, javac) y la estructura src/domain, src/presentation.

2. Estudien el diseño del programa: diagramas de paquetes y de clases.
   a. ¿Cuántos paquetes tenemos?

   R/=

   Tres paquetes: domain (modelo y fachada), presentation (interfaz gráfica) y test (pruebas de unidad).

   b. ¿Cuántas clases tiene el sistema?

   R/=

   En total 13 clases: 9 en domain (Content, Audiovisual, Movie, Episode, Series, Collection, NeoFlixException, NeoFlix y Log), 1 en presentation (NeoFlixGUI) y 3 en test (SeriesTest, EpisodeTest y NeoFlixTest). De la entrega original eran 10 archivos .java (8 de dominio, NeoFlixGUI y SeriesTest); NeoFlixException la creamos en la Parte I y las dos clases de pruebas nuevas las agregamos nosotros.

   c. ¿Cómo están organizadas?

R/=

Por capas y con dependencias unidireccionales: presentation depende de domain (la GUI usa la fachada NeoFlix) y test depende de domain (pruebas del modelo); domain no depende de nadie. Dentro de domain conviven el modelo (Content, Audiovisual, Episode, Series, Collection, Movie), la fachada (NeoFlix) y las clases de apoyo (NeoFlixException y Log).

d. ¿Cuál es la clase ejecutiva?

R/=

NeoFlixGUI: es la única clase con el método public static void main(String[] args). NeoFlix es la fachada del dominio (punto de acceso al modelo), no la clase ejecutiva.

3. Incluya los nuevos archivos en los directorios correspondientes para ejecutar el proyecto
   a. ¿Qué clases deben tener cada directorio?

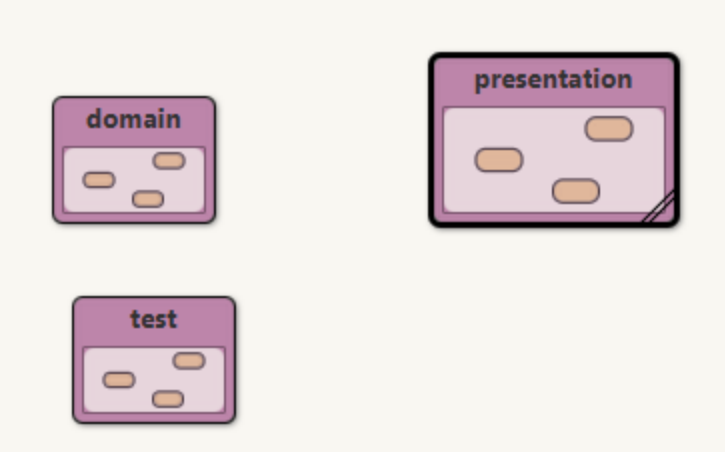
R/=

domain/: Content, Audiovisual, Movie, Episode, Series, Collection, NeoFlixException, NeoFlix y Log. presentation/: NeoFlixGUI. test/: SeriesTest, EpisodeTest y NeoFlixTest. Cada archivo coincide con la línea package que declara.

b. ¿Cómo los incluyeron?

R/=

Creamos la carpeta presentation/ dentro del proyecto y copiamos cada archivo .java a la carpeta de su paquete: NeoFlix.java y Log.java a domain/, y NeoFlixGUI.java a presentation/. Como el nombre de la carpeta coincide con la línea package, BlueJ los reconoce como paquetes y el proyecto recompila sin cambios adicionales.



4. Ejecute el proyecto
   a. ¿Qué funcionalidades ofrece?

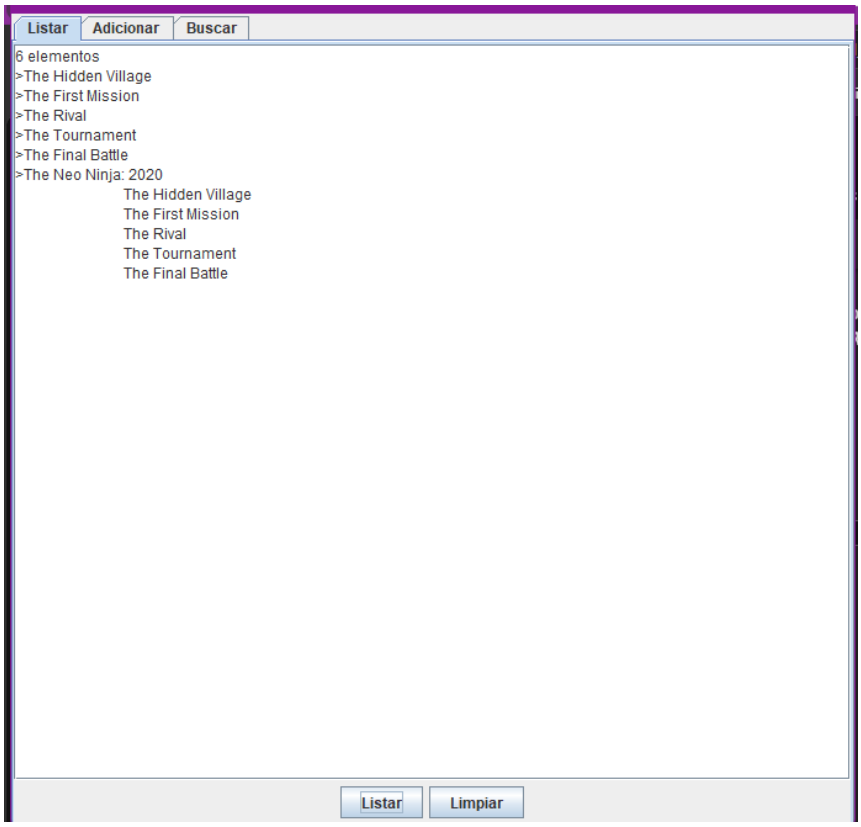
R/=

Tres pestañas: Listar (muestra todos los contenidos del catálogo), Adicionar (agrega un episodio —si el campo de episodios está vacío— o una serie con sus episodios) y Buscar (filtra por prefijo de título y muestra los resultados con indicadores).

   b. ¿Cuáles funcionan?

R/=

Listar funciona (muestra los 6 elementos iniciales). Adicionar funciona parcialmente: agrega sin validar, sin retroalimentación y sin limpiar campos; con datos no numéricos la consola muestra NumberFormatException. Buscar no funciona: cualquier búsqueda lanza IndexOutOfBoundsException por un defecto en select() (se documenta en la parte D).



5. Revisen el código y la documentación del proyecto.
   a. ¿De dónde salen los datos iniciales?

R/=

De NeoFlix.addSomeSeries(): método privado que invoca el constructor de NeoFlix y deja cargados los 5 episodios y la serie 'The Neo Ninja' (2020) con esos episodios, codificados directamente en el método como datos de ejemplo.

   b. ¿Qué clase pide que se adicionen?

R/=

Para los datos de ejemplo, el propio dominio: el constructor de NeoFlix pide adicionarlos llamando a addSomeSeries(). Para los datos que ingresa el usuario, la presentación: NeoFlixGUI.actionAdd() (botón [Adicionar]) lee los campos y pide al dominio adicionarlos.

   c. ¿Qué clase los adiciona?

R/=

NeoFlix (domain): addEpisode(...) crea el Episode y lo guarda en contents y en episodes; addSeries(...) crea la Series, le agrega los episodios indicados y la guarda en contents.

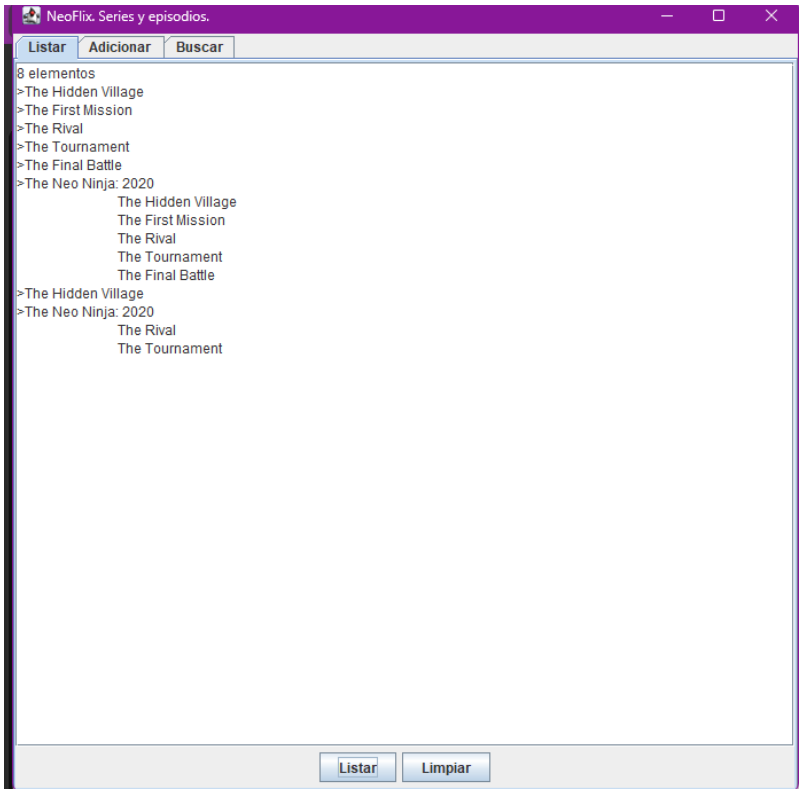
B Realizando inversa. Adicionar y listar. Todo OK.

1. Adicionen un nuevo episodio y un nueva nueva.
   Serie
   The Neo Ninja
   Episodio
   The Hidden Village
   a. ¿Qué ocurre?
   R/=

      Nada visible: no aparece ningún mensaje ni se limpian los campos. Internamente sí se agregan duplicados: el catálogo pasa de 6 a 8 elementos y episodes.put sobrescribe la clave 'THE HIDDEN VILLAGE' (el episodio anterior queda huérfano y consult() devuelve solo el primero).

   b. ¿Cómo lo comprueban? Capturen la pantalla.
   R/=

      Con la pestaña Listar → [Listar]: el listado ahora dice '8 elementos' y aparecen 'The Hidden Village' y 'The Neo Ninja' repetidos.



c. ¿Es adecuado este comportamiento?

R/=

No. El sistema acepta un nombre repetido sin avisar, deja el catálogo ambiguo (consult() devuelve solo el primero) y no da retroalimentación al usuario; se corregirá en la parte C.

2. Revisen el código asociado a adicionar en la capa de presentación y la capa de dominio.

a. ¿Qué método es responsable en la capa de presentación?

R/=

NeoFlixGUI.actionAdd() (listener del botón [Adicionar]): decide si es episodio o serie según el campo de episodios y llama al dominio.

b. ¿Qué método en la capa de dominio?

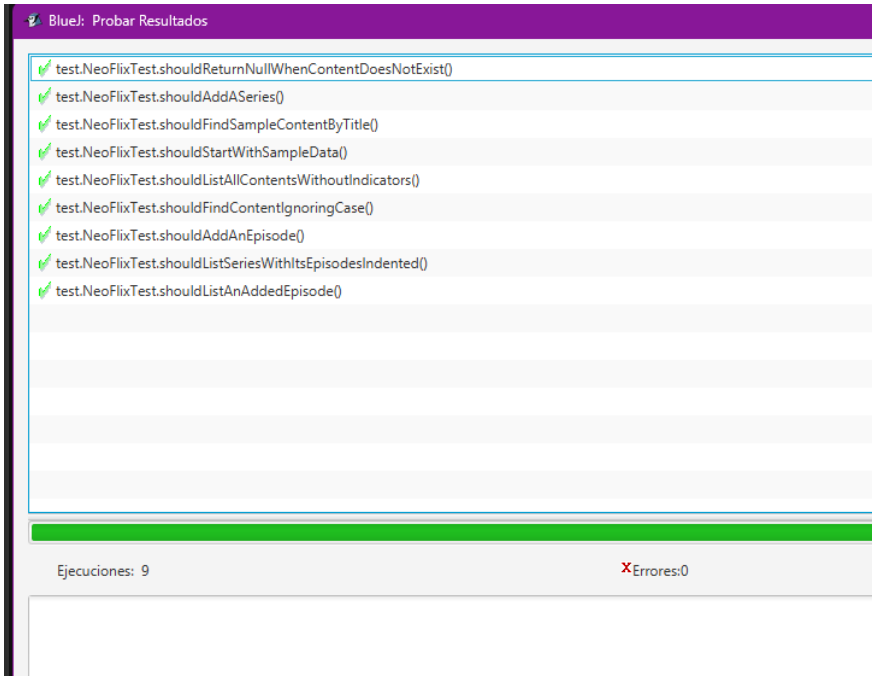
R/=

NeoFlix.addEpisode(...) y NeoFlix.addSeries(...).

3. Realicen ingeniería inversa para la capa de dominio para adicionar. Capturen los resultados de las pruebas de unidad.

R/=

Se creó test/NeoFlixTest.java con las pruebas N-01…N-06, que fijan el comportamiento actual válido (datos iniciales, consulta, adición de un episodio y de una serie).



4. Revisen el código asociado a listar en la capa de presentación y la capa de dominio.
   a. ¿Qué método es responsable en la capa de presentación?

   R/=

   NeoFlixGUI.actionList() (botón [Listar]): textDetails.setText(neoFlix.toString()).

   b. ¿Qué método en la capa de dominio?

   R/=

   NeoFlix.toString() → data(contents,false): encabeza con 'N elementos' y agrega la línea de cada contenido con Content.data(false) (Audiovisual.data o Series.data; en las series, sus episodios van indentados con tabulador).

5. Realicen ingeniería reversa para la capa de dominio para listar. Capturen los resultados de las pruebas de unidad.

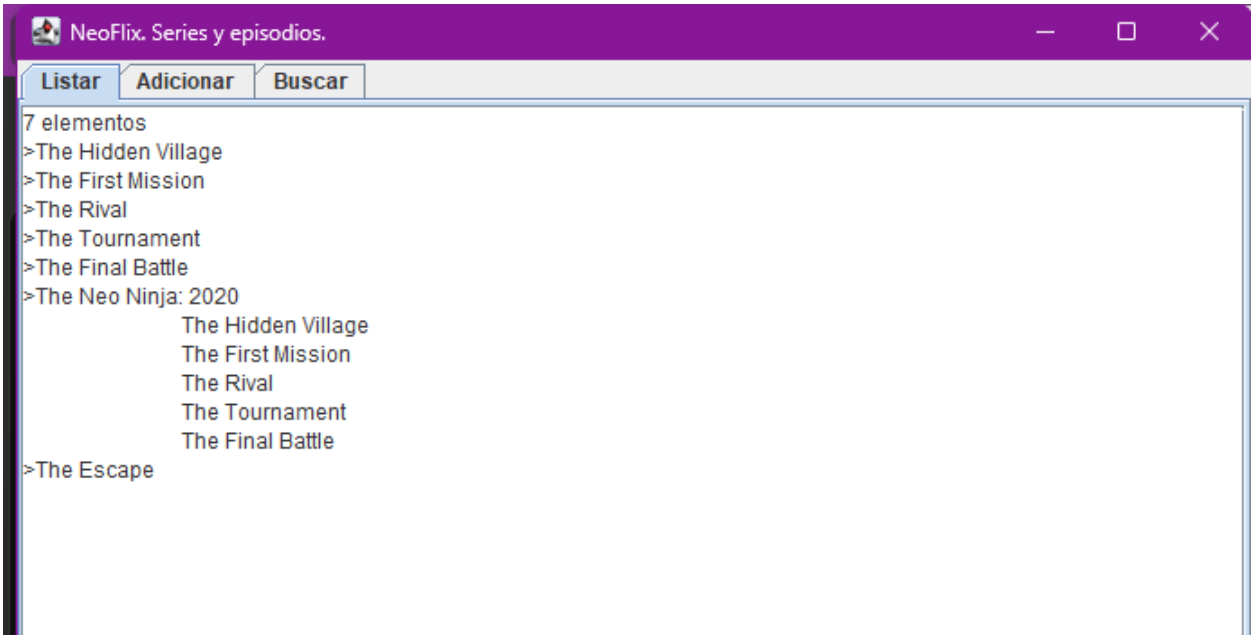
   R/=

   Con N-07…N-09 verificamos el listado: el texto exacto con los datos iniciales (conteo, formato '>', y la indentación con tabulador de los episodios de una serie) y el listado después de adicionar.

6. Propongan y ejecuten una prueba de aceptación. Capturen las pantallas correspondientes.

   R/=

   Prueba de aceptación (BDD) ejecutada sobre la GUI — A-01: dado el sistema recién iniciado, al pulsar [Listar] se ve '6 elementos' y la serie 'The Neo Ninja: 2020' con sus 5 episodios. — A-02: dado el catálogo inicial, al adicionar el episodio 'The Escape' (100, 80, 20, 150) y pulsar [Listar], se ve '7 elementos' y '>The Escape'.

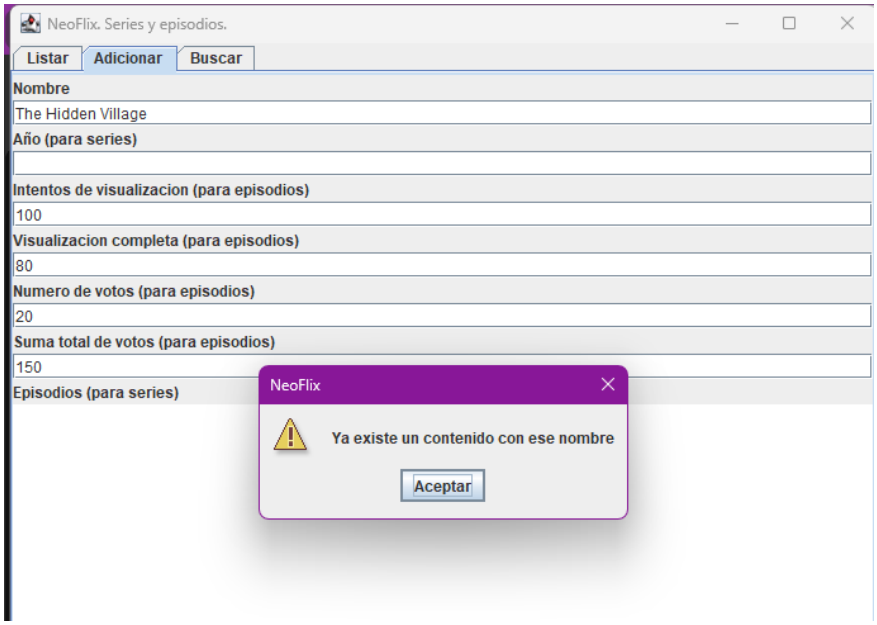


C Fortaleciendo. Adicionar robusta.

1. Propongan una prueba de aceptación que genere el fallo.
   a. ¿Se generó alguna excepción?

   R/=

   Se propuso una prueba de aceptación por cada regla: (a) título duplicado; (b) dato no numérico; (c) rangos inválidos (terminados > intentos; suma de votos > 10×votos) y (d) serie con un episodio inexistente. Con el código original NO se generaba ninguna excepción controlada: el duplicado se adicionaba en silencio; el dato no numérico lanzaba NumberFormatException sin controlar (visible solo en la consola de BlueJ); los rangos inválidos no fallaban al adicionar sino después, al calcular los indicadores; y el episodio inexistente dejaba la serie rota (NullPointerException al listar). Es decir, el fallo se presentaba tarde y lejos de su origen. Tras construir la solución (ver más abajo), cada caso produce el diálogo de advertencia con el mensaje de la NeoFlixException; la captura adjunta muestra el caso del título duplicado.



b. ¿Por qué?

R/=

Porque el dominio (NeoFlix.addEpisode/addSeries) no verificaba nada: confiaba en los datos, hacía Integer.parseInt directamente y no comprobaba unicidad, formato, rangos ni existencia de episodios; y la presentación tampoco atendía errores (no había try/catch ni mensajes). Sin validación en el punto correcto, la condición de error no se representa en ninguna parte: nadie la lanza, nadie la propaga y nadie la atiende; por eso el fallo aparecía después y lejos (indicadores rotos, listados que no responden, excepciones sin controlar en la consola). La solución convierte cada condición esperada en una NeoFlixException con mensaje claro, lanzada ANTES de modificar el catálogo y atendida en la GUI.

2. Analicen el diseño dinámico (diagrama de secuencia).Para hacer la funcionalidad robusta:

a. ¿Qué método debería lanzar la excepción?

R/=

NeoFlix.addEpisode(...) y addSeries(...): son los métodos que conocen las reglas del catálogo (título único, formato numérico, rangos coherentes, existencia de episodios) y los que deben lanzar NeoFlixException al detectar una violación.

b. ¿Qué métodos deberían propagarla?

R/=

Los métodos privados de validación del dominio en los que se apoyan (verificación del título nuevo, conversión a número y validación de los datos del episodio): declaran throws y dejan subir la excepción hasta los métodos públicos.

c. ¿Qué método debería atenderla? Expliquen claramente.

R/=

NeoFlixGUI.actionAdd() (presentación): es quien habla con el usuario; captura la NeoFlixException, muestra el mensaje con JOptionPane y no la propaga. No se registra en el log: es un error esperado del usuario.

3. Construya la solución propuesta. Incluyan las excepciones en el diseño y presente evidencia de las pruebas de unidad: una ejecución normal y una para cada excepción definida.

R/=

Solución construida: (1) en el dominio, addEpisode/addSeries validan en orden —título vacío, título repetido, datos no numéricos, valores negativos, terminados > intentos, suma de votos > 10×votos y episodio inexistente— y solo después modifican el catálogo (si algo falla, nada cambia); se apoyan en los métodos privados checkNewTitle, toNumber (traduce NumberFormatException y conserva la causa) y checkEpisodeData. (2) en la presentación, actionAdd atiende la NeoFlixException: muestra el mensaje en un diálogo de advertencia, no limpia los campos y no propaga la excepción (no se registra en el log: es un error esperado del usuario). Evidencia de pruebas de unidad: test/NeoFlixTest.java incluye una ejecución normal (adición válida, incluso en los límites 5/5/10/100 y 0/0/0/0) y una prueba por cada excepción definida (CONTENT_EXISTS, EMPTY_TITLE, NOT_NUMERIC, NEGATIVE_VALUE, INVALID_COMPLETED, INVALID_VOTES y EPISODE_NOT_FOUND); la batería completa del proyecto queda en verde.

4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1.
   a. ¿Qué sucede ahora? Capturen la pantalla.

R/=

Al repetir la aceptación, la aplicación responde de inmediato con el diálogo de advertencia que contiene el mensaje de la excepción —por ejemplo, «Ya existe un contenido con ese nombre» en el caso del duplicado (ver la captura incluida en esta sección)—; el formulario no se limpia (para que el usuario corrija) y el catálogo queda intacto: la aplicación sigue funcionando normalmente.

D Registrando errores. Consultar por patrones. ¡Error!

1. Consulten un contenido que inicia por T.
   a. ¿Qué sucede?

   R/=

   Con el defecto presente: en la pestaña Buscar no aparece ningún resultado (el área de resultados queda vacía, sin mensaje) y la ventana NO se cierra; en la consola de BlueJ se imprime la pila de un error. Sucede con cualquier prefijo, no solo con «T». Al final de esta parte, tras corregir el defecto, «T» muestra los 6 contenidos (con sus indicadores).

   b. ¿Qué error existe? Capturen el resultado.

   R/=

   Un java.lang.IndexOutOfBoundsException: Index 6 out of bounds for length 6, dentro de NeoFlix.select(): el ciclo recorre i = 0 hasta size-1 pero consulta contents.get(i+1); en la última vuelta el índice i+1 se sale de la lista. Además el filtro estaba mal planteado: evaluaba el elemento siguiente y agregaba el actual. La consola imprime la pila completa (NeoFlix.select → NeoFlix.search → NeoFlixGUI.actionSearch). El defecto quedó corregido al final de esta parte y las cinco pruebas de la consulta pasaron a verde.

   c. ¿Quién debe conocer y quien NO debe conocer esta información?

   R/=

   La información técnica del incidente (la pila del error) debe conocerla el equipo de desarrollo: queda en el registro (NeoSeries.log) para depurar. El usuario final NO debe verla: a él se le muestra un mensaje de alerta, sin detalles técnicos.

2. Revisen el código asociado a consultar en la capa de presentación y la capa de dominio.
   a. ¿Qué método es responsable en la capa de presentación?

   R/=

   NeoFlixGUI.actionSearch() (reacciona al escribir en el campo de [Buscar]).

   b. ¿Qué método en la capa de dominio?

   R/=

   NeoFlix.search(prefix), que delega la selección en NeoFlix.select(prefix) y el formateo en data(...).

3. Realicen ingeniería reversa para la capa de dominio para consultar. No modifiquen el código. Capturen los resultados de las pruebas de unidad.

   R/=

   Se escribieron cinco pruebas con el comportamiento CORRECTO esperado, sin modificar el código: selección por prefijo («The F» → 2), prefijo común («T» → 6), sin distinguir mayúsculas («the neo» → 1, la serie), sin coincidencias («X» → 0) y búsqueda con indicadores («The Neo» contiene «The Neo Ninja: 2020 (79 - 8)»). Con el defecto presente las cinco fallaron con IndexOutOfBoundsException (Tests run: 69, Failures: 5): ese reporte en rojo es la evidencia. Luego de corregir select() al final de la parte, las cinco pasaron y la batería completa quedó en verde (69 pruebas OK).

4. Exploren el método record de la clase Log
   a. ¿Qué servicio presta?

   R/=

   Presta el servicio de bitácora de errores: registra el incidente en el archivo NeoSeries.log (nivel SEVERE, con marca de tiempo y la pila de la excepción) para que el equipo lo investigue después; se usa para errores NO esperados.

5. Analicen el punto adecuado para que EN ESTE CASO se presente un mensaje especial de alerta al usuario, se guarde la información del error en el registro y continúe la ejecución. Expliquen y construyan la solución.

   R/=

   El punto adecuado es la PRESENTACIÓN (el borde del sistema), no el dominio: ni select() ni search() deben capturar la excepción (ocultarían el defecto y no saben hablar con el usuario). Solución construida: (1) en actionSearch, un try/catch general alrededor de neoFlix.search(...): al capturar, llama Log.record(e) (registro para el equipo técnico) y muestra en el área de resultados un mensaje de alerta sin detalles internos, y retorna normalmente para que la aplicación continúe (no se usa JOptionPane aquí porque la búsqueda se ejecuta con cada tecla del campo); y (2) red de seguridad global: en main se instala Thread.setDefaultUncaughtExceptionHandler, cuyo método handleUnexpected(error) registra en el log, muestra la alerta con JOptionPane y, si el error es un Error de la JVM (p. ej. OutOfMemoryError), termina la aplicación con System.exit(1); para una Exception, la aplicación continúa.

6. Ejecuten nuevamente la aplicación con el caso propuesto en 1.

a. ¿Qué mensaje salió en pantalla?

R/=

En el área de resultados aparece: «ALERTA: ocurrió un error inesperado y la búsqueda no pudo completarse. El equipo técnico fue notificado. Puede seguir usando NeoFlix.» No se muestran detalles técnicos: el usuario final no ve el tipo de excepción ni la pila.

b. ¿La aplicación termina?

R/=

No. La aplicación continúa y se puede seguir usando: la alerta queda en el área de resultados, el catálogo no se alteró (6 contenidos) y el usuario puede seguir operando. Solo se terminaría de forma controlada (System.exit(1)) si el error fuera un Error de la JVM o si el estado pudiera quedar inconsistente.

c. ¿Qué información tiene el archivo de errores?

R/=

NeoSeries.log (en el directorio desde donde se ejecuta) acumula una entrada por cada búsqueda fallida. Cada entrada tiene: fecha y hora, la clase y el método que la registraron (domain.Log.record), el nivel SEVERE, el mensaje de la excepción y su pila completa de llamadas (ArrayList.get → NeoFlix.select → NeoFlix.search → NeoFlixGUI.actionSearch). Primeras líneas de una entrada real: Oct 17, 2026 10:34:13 PM domain.Log record SEVERE: java.lang.IndexOutOfBoundsException: Index 6 out of bounds for length 6 (la fecha y hora se presentan según la configuración regional; en Windows en español, por ejemplo: «17/10/2026, 10:34:13 p. m.»).

7. Revisen el código

a. ¿Qué otro caso puede generar un error?

R/=

Los campos no numéricos: en actionAdd, la conversión Integer.parseInt (dentro de addEpisode/addSeries) lanza NumberFormatException (no controlada) si el usuario escribe texto o deja vacíos los campos numéricos.

b. ¿Cómo lo solucionarían?

R/=

Convertir y validar en el dominio: hacer la conversión a prueba de errores (capturar NumberFormatException y lanzar NeoFlixException con mensaje

claro) y atender en la presentación mostrando el mensaje; así un error esperado del usuario no rompe la consola.

8.
   a. ¿Es adecuado que la aplicación continúe su ejecución después de sufrir un incidente como este?

   R/=

   Sí, siempre que el sistema registre el incidente y avise al usuario: terminar abruptamente haría perder su trabajo; continuar con registro permite investigar y seguir operando.

   b. ¿De qué dependería continuar o parar?

   R/=

   De la naturaleza del error: si es esperado del usuario (dato inválido) → mensaje y continuar; si es inesperado (un defecto) → registrarlo en el log y continuar mientras el estado siga siendo consistente; detenerse solo si el error compromete los datos o el sistema.

9. Modifiquen la aplicación para garantizar que SIEMPRE que haya un error no esperado en presentación se maneje de la forma adecuada.
   a. ¿Cuál fue la solución seleccionada?

   R/=

   Dos piezas que concentran el comportamiento: (1) un try/catch específico en NeoFlixGUI.actionSearch: ante un error no esperado muestra la alerta EN LÍNEA (en el área de resultados, sin diálogo modal, porque la búsqueda se ejecuta con cada tecla), registra con Log.record y continúa; y (2) la red de seguridad GLOBAL: Thread.setDefaultUncaughtExceptionHandler instalado en main + handleUnexpected(Throwable), que registra el error en el log (envolviendo los Error en Exception para no cambiar la firma de Log.record), muestra la alerta con JOptionPane y solo termina la aplicación con System.exit(1) si es un Error de la JVM. Se seleccionó así porque concentra en un único punto el registro y la alerta, cubre todos los listeners presentes y futuros, y no duplica código de try/catch en cada uno.

PARTE III. DE BLUEJ A CONSOLA

A Comandos básicos del sistema operativo

1. Investiguen los comandos para moverse en la estructura de directorios: crear, borrar, listar su contenido y copiar o eliminar un archivo.

   R/=

   Para moverse: cd (entrar o ubicarse), cd .. (subir un nivel) y D: o cd /d D:\ruta (cambiar de unidad). Para listar: dir (Windows) o ls (Linux/macOS), y tree /f para el árbol completo. Para crear: mkdir (md). Para borrar: rmdir y del (archivos), con rd /s /q para carpetas con contenido. Para copiar: copy (archivo) y xcopy o robocopy (carpetas). Para mover o renombrar: move y ren (mv en Linux/macOS).

2. Organicen un nuevo directorio con la estructura propuesta para probar desde allí los comandos de consola. Consulten y capturen el contenido de su directorio.

   neoFlix

   src

   domain

   presentation

   Test

   R/=

   Se creó el directorio de trabajo con mkdir neoFlix y luego mkdir neoFlix\src\domain, neoFlix\src\presentation y neoFlix\src\test (en Linux/macOS: mkdir -p en una sola orden). La consulta con tree /f neoFlix muestra el árbol esperado: neoFlix → src → domain, presentation, test.

3. En el directorio src/domain copien únicamente el archivo Serie.java del paquete de dominio. Consulten y capturen el contenido de src/domain

   R/=

   Con copy RUTA\Series.java neoFlix\src\domain\ (cp en Linux/macOS) se copió únicamente ese archivo. La consulta de src/domain muestra un solo archivo, Series.java (en el enunciado aparece como Serie.java; la clase del proyecto se llama Series.java).

B Estructura de proyectos java

En java los proyectos se estructuran considerando tres directorios básicos.

neoFlix

src

bin

doc

lib

1. Investiguen los archivos que deben quedar en cada una de esas carpetas y la organización interna de cada una de ellas.
   R/=
   src/: TODOS los archivos .java, en subcarpetas por paquete (domain, presentation, test). bin/: los .class que genera javac, replicando la estructura de paquetes. doc/: la documentación HTML generada con javadoc (su página de entrada es index.html). lib/: las librerías de terceros en .jar (junit-4.13.2.jar y hamcrest-core-1.3.jar).

2.
   a. ¿Qué archivos de neoFlix deberían estar en el directorio src?
   R/=
   Todos los .java del proyecto con su paquete: los 9 de src/domain (Content, Audiovisual, Movie, Episode, Series, Collection, NeoFlix, NeoFlixException y Log), el de src/presentation (NeoFlixGUI.java) y los 3 de src/test (SeriesTest, EpisodeTest y NeoFlixTest). En bin, doc y lib no se guardan fuentes.

   b. ¿Cómo deberían estar organizados?
   R/=
   En subcarpetas cuyo nombre coincide con el paquete declarado en cada archivo: src/domain/, src/presentation/ y src/test/. Así lo exigen javac y la JVM para localizar las clases.

3. Copien a src únicamente los archivos de los paquetes de presentación y dominio. Consulten y capturen el contenido de los directorios que se modificaron.
   R/=
   Se copiaron a src/domain los 9 archivos del paquete de dominio y a src/presentation NeoFlixGUI.java; las pruebas se agregaron después a src/test. El listado de los directorios (dir /s o ls -R) confirma 9 archivos .java en src/domain y 1 en src/presentation.

C Comandos de java

1. Consulten para qué sirven cada uno de los siguientes comandos:
   javac
   java
   javadoc
   Jar
   R/=

   javac: compilador; traduce los .java a bytecode .class. java: lanzador; arranca la JVM y ejecuta una clase con main (o un .jar con -jar). javadoc: genera la documentación API en HTML a partir de los comentarios de documentación del código. jar: empaquetador; crea, consulta y extrae archivos .jar (bytecode + manifiesto).

2. Creen una sesión de consola y consulten en línea las opciones de los comandos java y javac. Capturen las pantallas.
   R/=

   Con javac -help y java -help (o --help) la consola imprime el resumen de opciones de cada herramienta: javac muestra, entre otras, -d (carpeta de salida), -cp (classpath), -encoding y -sourcepath; java muestra -cp, -jar y -version. Con -X se ven las opciones adicionales.

3. Busquen la opción que sirve para conocer la versión a que corresponden estos dos comandos. Documenten el resultado.
   R/=

   La opción de versión es --version (o -version): se usaron javac --version y java --version. El resultado documentado en el equipo de trabajo fue JDK 21 (javac 21.0.12.1 y openjdk 21.0.12.1): ambas versiones coinciden, lo que confirma que el PATH apunta al JDK correcto.

D Compilando

1. Utilizando el comando javac, desde el directorio raiz (desde neoFlix con una sola instrucción), compile el proyecto.
   a. ¿Qué instrucción completa tuvo que dar a la consola para compilar TODO el proyecto? Tenga presente que se pide un único comando y que los archivos compilados deben quedar en los directorios respectivos.

R/=

javac -d bin src\domain*.java src\presentation*.java (una sola instrucción, desde neoFlix; en Linux/macOS las rutas usan /). javac compila ambos paquetes y -d deja cada .class en bin/, dentro de la carpeta de su paquete.

2. Revise de nuevo el contenido del directorio de trabajo y sus subdirectorios.
   a. ¿Cuáles nuevos archivos aparecen ahora y dónde se ubican? Capturen las pantallas con los contenidos de los directorios de neoFlix/bin.

R/=

Aparecen 9 archivos .class en bin\domain (uno por clase del dominio) y 8 en bin\presentation: NeoFlixGUI.class y sus clases internas anónimas NeoFlixGUI$1.class a $7.class (listeners de los botones y del campo de búsqueda). Los .java permanecen intactos en src.

E Documentando

1. Utilizando el comando javadoc, desde el directorio raiz, genere la documentación (API) en formato html, en este directorio.
   a. ¿Cuál es el comando completo para generar esta documentación?

R/=

javadoc -d doc -sourcepath src -subpackages domain:presentation -encoding UTF-8 -docencoding UTF-8 -charset UTF-8 (desde neoFlix). La opción -d dirige la salida al directorio doc y -subpackages procesa los paquetes domain y presentation.

2.
   a. ¿Cuál archivo hay qué abrir para empezar a navegar por la documentación? Ábranlo y capturen la pantalla.

R/=

doc\index.html: es la página de entrada del API; desde ella se navega por los paquetes (domain y presentation) y sus clases.

F Ejecutando

1. Empleando el comando java, desde el directorio raíz, ejecute el programa.
   a. ¿Cómo utilizó este comando?

R/=

java -cp bin presentation.NeoFlixGUI: con -cp bin se indica dónde quedaron los .class y luego el nombre completo de la clase (con su paquete), que es la que tiene el método main.

2. Capturen la pantalla del programa en ejecución.

R/=

La ejecución abre la ventana de NeoFlix con sus tres pestañas (Listar, Adicionar y Buscar) y los datos de ejemplo, a la espera de la interacción del usuario (la evidencia de la interfaz en ejecución es la misma ventana de las capturas de la Parte II).

G Probando

1. Adicione ahora los archivos del paquete de pruebas y trate de compilar nuevamente el programa.
   a. ¿Qué pasa?

R/=

La compilación falla con errores del tipo package org.junit does not exist y cannot find symbol en las clases de prueba: el compilador no encuentra las clases de JUnit porque la librería todavía no está en el classpath.

2. Tenga en cuenta que estas clases requieren la librería junit.
   a. ¿Dónde se incluye una librería en la estructura de archivos?

R/=

En lib/, junto al resto de librerías de terceros en .jar. Se usaron junit-4.13.2.jar (framework de pruebas) y hamcrest-core-1.3.jar (su dependencia), tomadas de Maven Central.

   b. ¿Cómo se compila incluyendo un .jar?

R/=

Agregando al comando la opción -cp (classpath) con la carpeta de las librerías: -cp "lib/*" (el asterisco incluye todos los .jar de lib). Alternativa explícita en Windows: -cp "lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" (en Linux/macOS el separador es : y las rutas con /).

   c. ¿Qué instrucción completa tuvo que dar a la consola para compilar?

R/=

javac -d bin -cp "lib/*" src\domain*.java src\presentation*.java src\test*.java (Windows; en Linux/macOS igual con /). Compila dominio, presentación y pruebas, y deja los .class en bin/.

3. Ejecute desde consola las pruebas.
   a. ¿Cómo utilizó el comando java?.

   R/=

   java -cp "bin;lib/*" org.junit.runner.JUnitCore test.SeriesTest test.EpisodeTest test.NeoFlixTest: se ejecuta la clase org.junit.runner.JUnitCore (el ejecutor de JUnit 4) dándole como argumentos los nombres completos de las tres clases de prueba, con el classpath formado por bin y las librerías. (En Linux/macOS el separador del classpath es :).

4. Pegue en su documento el resultado de las pruebas

   R/=

   El resultado impreso por la consola fue: OK (69 tests). Las tres suites pasan: 25 pruebas en SeriesTest, 10 en EpisodeTest y 34 en NeoFlixTest.

H Empaquetando

1. Consulten como utilizar desde consola el comando jar para empaquetar su programa entregable en un archivo .jar, que contenga los archivos bytecode necesarios (no las fuentes ni las clases de prueba)
   a. ¿Cómo empaquetó jar ?

   R/=

   El comando consultado para empaquetar desde consola es: jar cfe neoFlix.jar presentation.NeoFlixGUI -C bin domain -C bin presentation (c = crear, f = nombre del archivo, e = clase principal; -C bin domain añade solo esas carpetas de bytecode, sin fuentes ni pruebas). Para el entregable, el archivo .jar se obtiene desde la interfaz de BlueJ con Project → Create Jar File..., seleccionando NeoFlixGUI como Main Class.

2.
   a. ¿Cómo se ejecuta el proyecto empaquetado? Capturen la pantalla del programa en ejecución.

   R/=

Con java -jar neoFlix.jar (o con doble clic sobre el archivo). La JVM lee el manifiesto, ubica la clase principal NeoFlixGUI y abre la aplicación; el registro NeoSeries.log se crea en la carpeta desde donde se ejecuta.

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

   R/=

   16 horas por integrante (José García y Kevin Quezada), distribuidas entre el desarrollo de las Partes I y II, las pruebas, las evidencias y la elaboración del informe.

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

   R/=

   Completo y funcional: las tres pestañas operan, el manejo de excepciones está implementado y cubierto por 69 pruebas en verde, la documentación API se genera desde consola y el defecto de la búsqueda quedó corregido con su evidencia. Queda como cierre el empaquetado .jar desde BlueJ.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

   R/=

   La refactorización: extraer responsabilidades con nombres claros (findSeries, clearAddForm, toNumber) mantuvo el código simple y permitió corregir el defecto de select() cambiando una única idea. La simplicidad del diseño (construir solo lo pedido) evitó deuda técnica.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

   R/=

   El cierre del manejo de errores: el dominio lanza excepciones con mensaje claro y de forma atómica (si algo falla, nada cambia), la presentación las atiende sin romper la aplicación y la red de seguridad global registra lo imprevisto. Es el mayor logro porque convierte un sistema frágil en uno predecible, y quedó cubierto por 69 pruebas automatizadas.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

   R/=

   El defecto de select(): toda búsqueda lanzaba IndexOutOfBoundsException. Lo aislamos escribiendo cinco pruebas desde el comportamiento esperado (evidencia en rojo), revisamos la bitácora y corregimos la causa (el índice i+1) sin tocar el síntoma; con eso las pruebas pasaron a verde.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

   R/=

   Bien: avanzamos por bloques con las pruebas en verde antes de continuar y documentamos mientras construíamos. Compromiso de mejora: registrar las evidencias (capturas y bitácora) al cierre de cada sesión y revisar el entorno (JDK y classpath) al inicio de cada una.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

   R/=

   Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley. · Oracle. (s. f.). Lesson: Exceptions. The Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/exceptions/ · Oracle. (s. f.). Documentación de las herramientas javac, java, javadoc y jar (JDK Tools Reference). https://docs.oracle.com/en/java/javase/21/docs/specs/man/ · JUnit. (s. f.). JUnit 4. https://junit.org/junit4/

   La más útil fue el capítulo de excepciones de Effective Java: de allí salieron las reglas que guiaron el diseño (excepciones verificadas para lo recuperable, atomicidad al adicionar y no ignorar excepciones).